

Multi-Agent Reinforcement Learning for No-Limit Texas Hold'em

Aléo Ibrahim Houmed & Maxence Adly

Reinforcement Learning Group Project

1. Introduction & Contributions

No-Limit Texas Hold'em (NLHE) presents a massive AI challenge due to imperfect information, stochastic dynamics, continuous bet sizing, and sparse delayed rewards. We train autonomous agents via **self-play reinforcement learning**, where a shared policy improves by competing against itself.

Key Contributions:

- Custom multi-agent environment (2–10 players, cash-game dynamics).
- PPO implementation with strict **action masking** for legal moves.
- Interactive evaluation tools (GUI & CLI) for real-time play.

2. Game & Environment Design



Observation Space (normalized to $[0, 1]$):

- Hole & Board cards (one/multi-hot)
- Street indicator & Position
- Stacks, pot, bets, min/max raises

Action Space (6 discrete actions): Fold, Check/Call, Raise Min, Raise Half Pot, Raise Pot, All-In.

Reward: Zero-sum at hand end: $r_i = \text{stack}_i^{\text{final}} - \text{stack}_i^{\text{initial}}$

3. Method: PPO with Action Masking

We use **Proximal Policy Optimization (PPO)** within the Ray RLLib framework, employing a single shared policy (*self-play*).

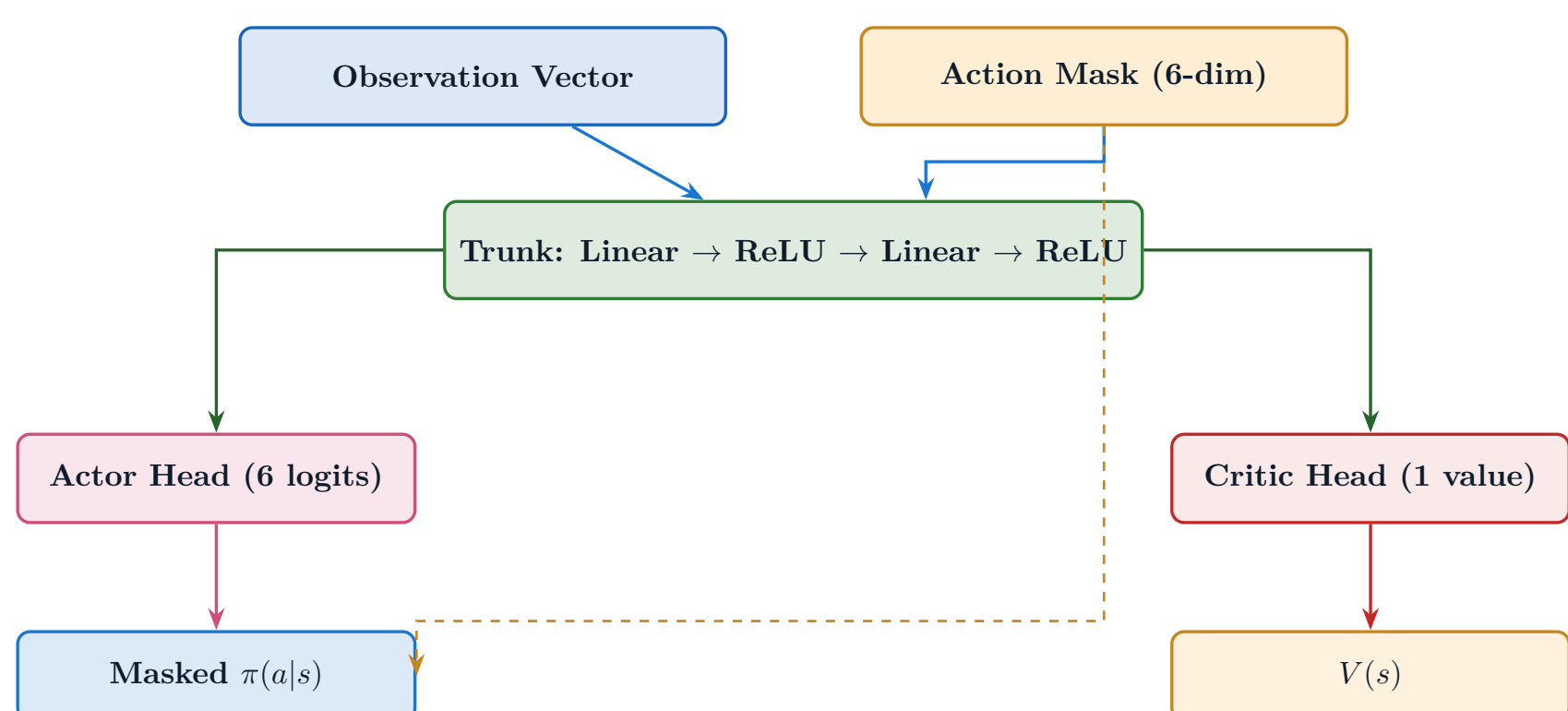
Action Masking: Illegal actions are completely suppressed by setting their logits to -10^{38} before the softmax layer, guaranteeing $\pi(a|s) = 0$ for invalid moves:

$$\ell'_i = \begin{cases} \ell_i & \text{if action } i \text{ is legal} \\ -10^{38} & \text{otherwise} \end{cases}$$

Key Hyperparameters:

Hyperparameter	Value
Learning Rate	3×10^{-4}
Discount γ	1.0 (episodic)
GAE λ	0.95
Clip parameter ϵ	0.2
Entropy coefficient	0.02
Train batch size	4096

4. Neural Network Architecture



- Shared trunk: 2-layer MLP ($d \rightarrow 256 \rightarrow 256$) with ReLU
- Actor head: $\text{Linear}(256, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, 6)$, masked
- Critic head: $\text{Linear}(256, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, 1)$

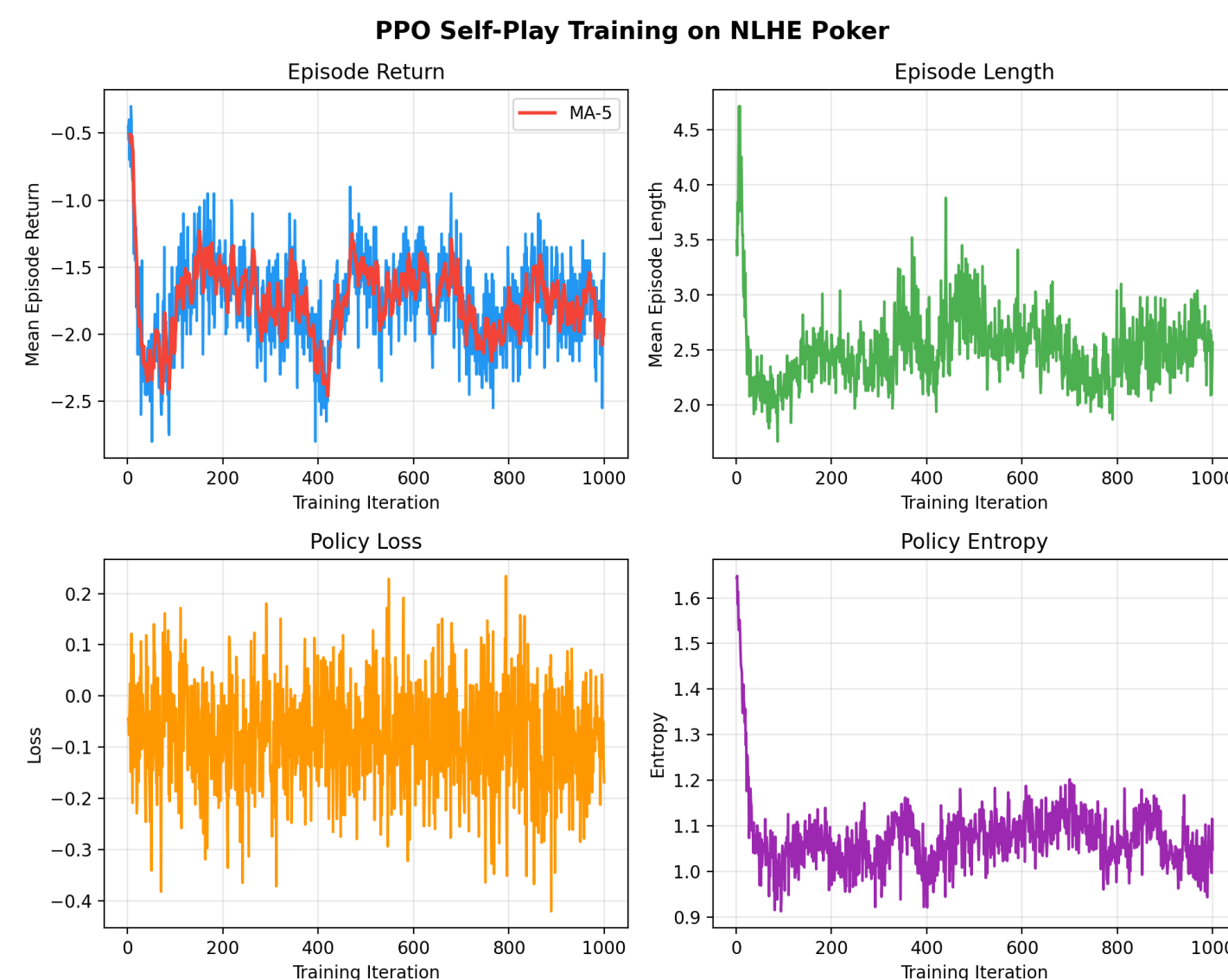
8. Implementation & Future Work

Tech Stack: Ray RLLib 2.54 (RLModule API), PyTorch 2.10, Custom Gymnasium MultiAgentEnv, Treys library for hand evaluation.

Future Directions:

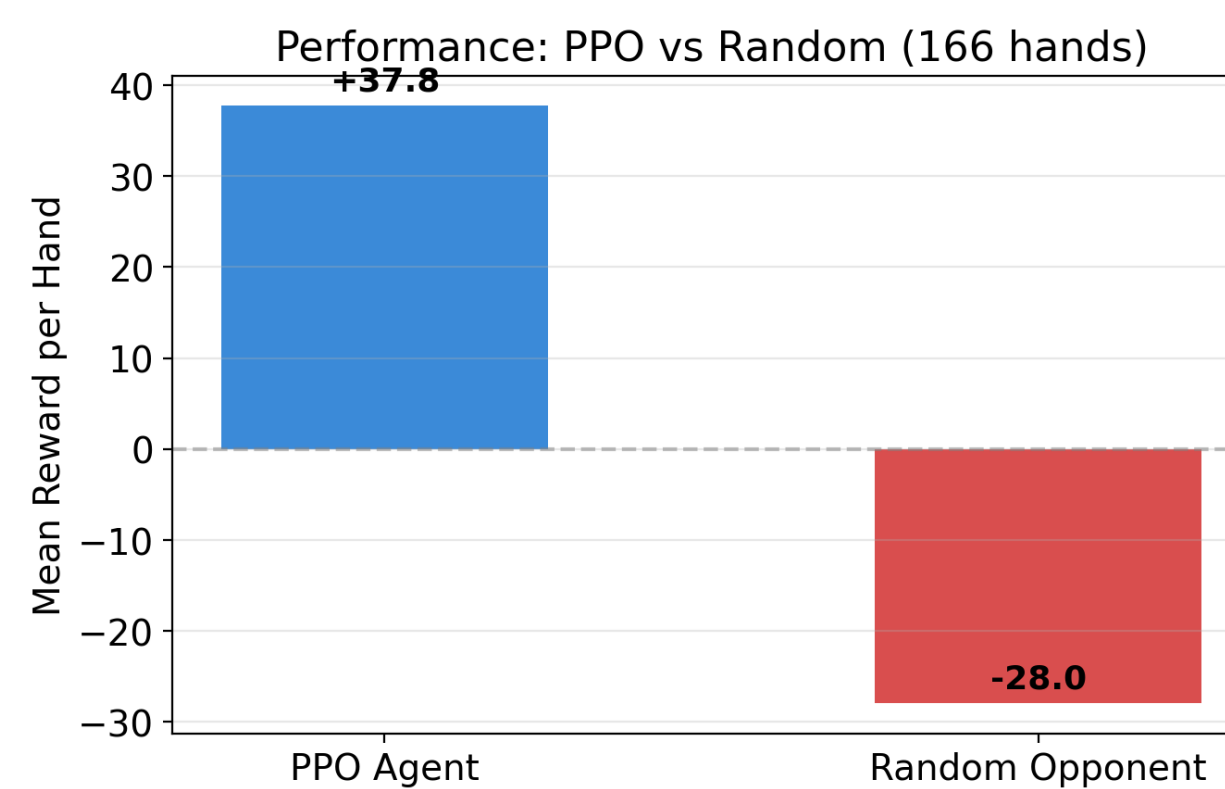
- Scale to 6–10 player tables and longer training horizons.
- Implement **population-based training** to avoid strategy collapse.
- Add **opponent modelling** and continuous bet sizing.

5. Training Results



- Episode return** oscillates around zero, expected in **zero-sum self-play**.
- Episode length** stabilises around 2–3 steps (many hands resolve pre-flop).
- Entropy** decreases from ~ 1.6 to ~ 1.1 as the policy becomes decisive.
- Scale:** 4M environment steps across 1000 PPO iterations.

6. Evaluation vs Random Opponent

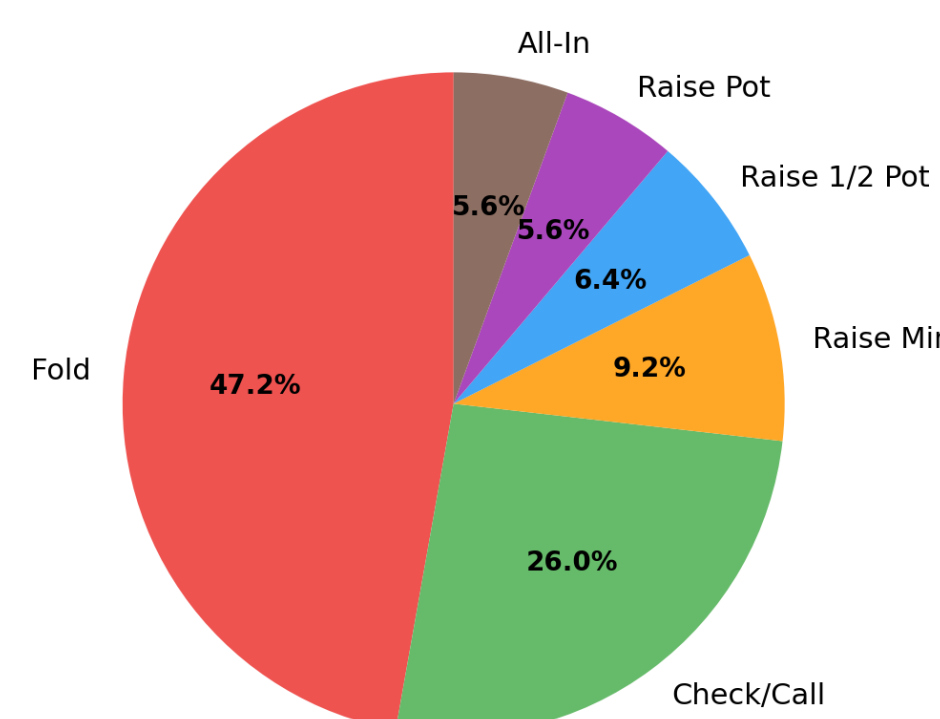


Metric	PPO Agent	Random
Mean reward/hand	+37.8	-28.0
Win rate (reward > 0)	21.1%	-

The PPO agent significantly outperforms a random baseline (+37.8 chips/hand). Despite a modest win rate, the agent wins *larger pots*, extracting value from strong hands.

7. Learned Action Distribution

Trained Agent Action Distribution



The trained policy exhibits **tight-aggressive** characteristics recommended in theory:

- Fold (47.2%)**: High selectivity; avoids chasing weak hands.
- Check/Call (26.0%)**: Pot control with marginal holdings.
- Raise / All-In (26.4%)**: Willingness to build pots with strong hands.